

Axona API Reference

Reference for every public symbol exported from @axona/protocol v2.8.0. Organized by what application developers actually reach for; deep-customization surfaces are at the end.

Companion documents:

- Quick Start — 5-minute working roundtrip.
- Programmer Guide — mental model + worked

example + pitfalls.

- Security changelog — what each kernel

version protects (authenticated handshake, channel binding, pub/sub trust boundary, verified routing admission). See §17 below for the developer-facing summary of the security model.

Imports throughout assume:

```
import { /* ... */ } from '@axona/protocol';
```

Sub-path imports (e.g. @axona/protocol/contracts/Transport.js) work for symbols that need it but most apps only need the main barrel.

Table of contents

Application surface (what you'll use first)

1. Identity
2. AxonaPeer construction + lifecycle
3. Pub/sub
4. Direct messaging
5. Mesh introspection
6. Topic IDs
7. Envelopes + verification
8. Errors
9. Persistence

Transport + protocol surface

10. Contracts
11. Sim transport
12. Handshake

Low-level utilities

13. Ed25519 helpers

- 14. Hex / ID math
- 15. Geo utilities
- 16. Constants

Types referenced throughout

Identity, Envelope, Subscription, Peer, TopicId.

Application surface

Types

Identity (type) {#identity-type}

```
{
  id:          string;           // 66-char lowercase hex (264-bit nodeId)
  pubkey:      Uint8Array;       // 32-byte Ed25519 public key
  pubkeyHex:   string;           // 64-char lowercase hex
  privateKey:  CryptoKey;        // Web Crypto Ed25519 (browser + Node 20+)
  region:      { lat: number, lng: number };
  createdAt:   number;           // ms epoch
}
```

The top 8 bits of `id` are `geoCellId(lat, lng, 8)` (the S2 cell at level 8); the bottom 256 bits are `sha256(pubkey)`.

Envelope (type) {#envelope-type}

```
{
  msgId:       string;           // 64-char hex sha256 of canonical inputs
  ts:          number;           // ms epoch when published
  topic:       string;           // the application-level topic name
  message:     any;              // any JSON-serializable payload
  signerPubkey?: string;         // 64-char hex; present iff signed
  signature?:  string;           // "ed25519:<128 hex>"; present iff signed
}
```

Subscription (type) {#subscription-type}

Opaque handle returned by `peer.sub()`. Has:

```
sub.id         // string – unique per subscription
sub.topicId    // string – 66-char hex topic ID
sub.topicName  // string – your original topic name
sub.stop()     // Promise<void> – cancel; idempotent
```

PeerId (type) {#peer-id-type}

In the kernel's public API, peer IDs are **66-char lowercase hex strings**. The synaptome stores them as `bigint` internally; conversion happens at the API boundary. Pass strings unless a function's signature specifically says `BigInt` (`peer.lookup`, the synapse admit example).

TopicId (type) {#topic-id-type}

Same shape as `PeerId` — 66-char lowercase hex. Top 8 bits encode the addressing-mode S2 prefix, bottom 256 bits are sha256 of the canonical input.

1. Identity

```
deriveIdentity({ lat, lng, extractable? }) → Promise<Identity>
```

Generate a fresh 264-bit Ed25519 identity. Top 8 bits of the resulting `id` encode the S2 cell containing (`lat`, `lng`); the bottom 256 bits are SHA-256(`pubkey`), so the `nodeId` is **self-authenticating** — a peer can only claim an `id` it holds the private key for (this is what the axona/4 handshake checks; see §17).

```
const id = await deriveIdentity({ lat: 38.0, lng: -77.0 });
console.log(id.id);           // 'df7c5e...' (66 hex chars, us-east prefix)
console.log(id.id.slice(0, 2)); // 'df'      (us-east S2)
```

Optional `extractable` (default `true`): pass `false` to hold the signing key as a **non-extractable** `CryptoKey` — it can sign but cannot be read back out by page script, so an XSS incident or a malicious dependency cannot exfiltrate the long-term identity. Recommended for ephemeral browser identities. Leave `true` only when you need to `dumpIdentity` it for persistence.

Returned `Identity`: { `id` (66-hex), `pubkey` (`Uint8Array`), `pubkeyHex` (64-hex), `privateKey` (`CryptoKey`), `region`, `createdAt`, `sign`(bytes), `verify`(bytes, sig) }.

Throws `IdentityError` if Web Crypto isn't available or `lat`/`lng` are out of range.

```
dumpIdentity(identity) → Promise<envelope>
```

Serialize an `Identity` to a JSON-safe envelope (the `privateKey` is exported as base64 PKCS#8).

```
const env = await dumpIdentity(identity);
// {
//   id:      '<66 hex>',
//   pubkey:  '<64 hex>',
//   privkey: '<base64 PKCS#8 string>',
//   region:  { lat, lng },
//   createdAt: <ms>,
// }
localStorage.setItem('my-identity', JSON.stringify(env));
```

```
loadIdentity(envelope) → Promise<Identity>
```

Inverse of `dumpIdentity`. Reconstructs the full `Identity` from the JSON envelope by importing the PKCS#8 key.

```
const env = JSON.parse(localStorage.getItem('my-identity'));
const identity = await loadIdentity(env);
```

Throws `IdentityError` (`IDENTITY_LOAD_FAILED`) if the envelope is malformed or the `pubkey` doesn't match the `id`'s hash.

2. AxonaPeer construction + lifecycle

The peer is the per-node DHT contract implementation.

```
new AxonaPeer({ domain, node, identity, transport?, axonaManager?, persist?, engine?
})
```

Construct a peer. Required:

- `domain` — an `AxonaDomain` instance (shared routing parameters).
- `node` — a `NeuronNode` (or any object with `{id, alive, transport, synaptome}`).
- `identity` — from `deriveIdentity`; needed for signed publishes.

Optional:

- `transport` — overrides `node.transport`. The peer uses it directly for send/notify/lookup round-trips.

- `axonaManager` — pre-built `AxonaManager`. If omitted, the peer resolves one on first pub/sub via `engine.axonaManagerFor(node)`.

- `persist` — a `PersistenceAdapter`; enables auto-checkpointing of identity envelope, subscriptions, and synaptome snapshots.

- `engine` — legacy; pass null for new code.

```
const peer = new AxonaPeer({
  domain:    new AxonaDomain({ k: 20 }),
  node:      new NeuronNode({ id: BigInt('0x' + id.id), lat, lng }),
  identity:  id,
  transport,
});
```

`peer.start() → Promise<void>`

Bring the peer up — installs wire-level routing handlers, sets up the delivery hook, and (if persistence is wired) hydrates any persisted state.

Idempotent. Returns the peer once started.

`peer.stop() → Promise<void>`

Tear down. Drains in-flight pub calls, stops timers, releases listeners. Idempotent.

`peer.join(opts?) → Promise<void>`

Production bootstrap: discover the network, populate synaptome to target size, ready for routing. Optional `opts` customize fan-out parameters; defaults are usually fine.

In simple in-process setups (where you admit synapses directly), you don't need `join()` — `start()` is enough.

`peer.leave({ drain?, notify?, timeoutMs? }) → Promise<void>`

Graceful shutdown. With `drain: true` (default), waits up to `timeoutMs` (default 5000) for in-flight pubs to complete. With `notify: true` (default), sends `pubsub:unsubscribe-k` for every active subscription so axons can sweep our entries promptly.

```
await peer.leave({ drain: true, notify: true, timeoutMs: 3000 });
```

`peer.getNodeId() → string | bigint`

Returns whatever was passed as `node.id` at construction time. If you followed the convention of passing a `BigInt` to `NeuronNode`, this returns `BigInt`; if you passed a string, it returns a string.

3. Pub/sub

`peer.pub(topicName, message, opts?) → Promise<msgId>`

Publish a message to a topic.

Arg	Type	Notes		
topicName	string	Application-defined name (gets hashed).		
message	any	JSON-serializable. Strings, numbers, objects, arrays all OK. Size: see below.		
opts.publisher	`string`	null	undefined	Addressing mode. See §6.
opts.sign	boolean	Default true. Set false for anonymous publishes (no signerPubkey/ signature in envelope).		

Returns: 64-char hex `msgId` — sha256 of the canonical envelope inputs. Subscribers see this same id in `env.msgId`.

Maximum message size: 256 KiB (kernel $\geq$ v2.8.1; was 64 KiB before). The limit is on the *serialized envelope* (`JSON.stringify({ msgId, ts, topic, message, signature, signerPubkey })`), so your usable payload is 256 KiB minus a few hundred bytes of envelope overhead. It's measured on string length, so heavily multi-byte (CJK/emoji) content can exceed 256 KiB on the wire.

The cap is currently enforced **at the receiving root axon**, not at `peer.pub`: an oversized publish is **silently dropped network-side** (logged `publish-oversize-dropped`) — the publisher gets **no error**. So if a large message "doesn't arrive," check its size first.

Pub/sub is a *broadcast* path (replicated to K root axons, cached, fanned to all subscribers), so it is the wrong place for images/documents even under 256 KiB. For binary content, publish a small **content-reference** (hash + size + mime) and transfer the bytes out-of-band, content-addressed.

```
const synth = '<region S2 hex>' + '0'.repeat(64);
const msgId = await peer.pub('news/world', { title: 'hi' }, {
  publisher: synth,
});
```

Throws `PublishError`:

- `PUBLISH_INVALID_TOPIC` — empty topic name.
- `PUBLISH_SIGN_FAILED` — signing failed (identity missing privateKey, Web Crypto unavailable, etc).

- `PUBLISH_PAYLOAD_TOO_LARGE` — the message is **not JSON-serializable** (circular reference, `BigInt`, etc.), so the envelope can't be encoded. Despite the name, this does **not** fire on size — there is no client-side size check today; oversized publishes are dropped at the receiving

root axon instead (see "Maximum message size" above).

`peer.sub(topicName, handler, opts?) → Promise<Subscription>`

Subscribe to a topic. handler is invoked for every delivered envelope.

Arg	Type	Notes			
topicName	string	Must match what publishers use.			
handler	(envelope) => void	Called with the full Envelope.			
opts.publisher	`string` \	null \	undefined`	Must match publisher's mode.	
opts.since	`all` \	'latest' \	number \	undefined`	Replay mode. See below.

since **modes**:

since	What you get
omitted / undefined	Live tail only — no cached messages.
'all'	Every message in the axon's replay cache, then live tail.
'latest'	The most recent ~1s of cache, then live tail.
<number>	Messages with publishTs > since (ms epoch).

Returns: a Subscription handle. Call `.stop()` to cancel.

```
const sub = await peer.sub('news/world', (env) => {
  console.log(env.ts, env.message);
}, { publisher: synth, since: 'all' });

// later...
await sub.stop();
```

Throws SubscribeError:

- SUBSCRIBE_INVALID_TOPIC
- SUBSCRIBE_HANDLER_MISSING

`peer.pull(msgId, opts) → Promise<Envelope | null>`

Fetch a specific envelope by its msgId from the K-closest axons. Returns null if it's not in any reachable cache window.

Arg	Notes
msgId	64-char hex from pub or a previous delivery.
opts.topic	The topic name.
opts.publisher	Same addressing mode used to publish.
opts.timeoutMs	Default 1000.

```
const env = await peer.pull(msgId, {
  topic: 'news/world',
  publisher: synth,
  timeoutMs: 1000,
});
if (env) console.log('found:', env.message);
```

Throws `PullError` on transport failure (not on cache miss — that returns `null`).

`peer.metrics(topicName, opts?) → Promise<metricsObj>`

Best-effort delivery + subscriber counts for a topic.

```
const m = await peer.metrics('news/world', {
  publisher: synth,
  timeoutMs: 500,
});
// {
//   publishes: <distinct messages seen>,
//   subscribers: <approx active subscribers>,
//   reshare_count: <count>,
// }
```

Values are aggregated from whichever axons reply within `timeoutMs`. Use for UX ("12 people in this room"), not for billing.

4. Direct messaging

For 1:1 messages between specific peers, bypassing pub/sub.

`peer.send(peerIdHex, message) → Promise<reply>`

Request/response. Resolves to whatever the recipient's `onMessage` handler returns (or rejects with the remote handler's thrown error).

```
const reply = await alice.send(bobIdHex, {
  kind: 'ping', body: 'hello',
});
// reply = whatever bob's handler returned
```

Throws `TypeError` if `peerIdHex` isn't 66-char hex.

`peer.notify(peerIdHex, message) → Promise<boolean>`

Fire-and-forget. Resolves once the frame is enqueued (NOT when delivered). Return value is `true` on successful enqueue, `false` if the transport rejected it.

```
await alice.notify(bobIdHex, { kind: 'typing', user: 'alice' });
```

`peer.onMessage(handler) → void`

Register the single direct-message handler. Calling `onMessage` twice **replaces** the previous handler — for multi-kind dispatch, do it inside the handler:

```
peer.onMessage(async (senderIdHex, message) => {
  switch (message?.kind) {
    case 'ping':
      return { reply: 'pong' }; // send() resolves to this
    case 'typing':
      console.log(`${senderIdHex} typing`);
      return; // notify() discards
```

```

    }
  });

```

senderIdHex is set by the transport at the receiving end. It's the authenticated sender (per the transport's binding); for higher assurance verify a signed envelope inside message.

5. Mesh introspection

peer.peers() → string[]

Current synaptome membership as 66-char hex strings.

peer.onPeerJoin(handler) → () => void

Fires when a peer is admitted to the synaptome. Handler receives (peerIdHex, ctx). Returns an unsubscribe function.

```

const unsub = peer.onPeerJoin((peerId, ctx) => {
  console.log('admitted', peerId, 'via', ctx?.addedBy);
});
// later: unsub();

```

peer.onPeerLeave(handler) → () => void

Symmetric. Fires when a peer is evicted (TTL, churn, explicit unbind). Same signature.

peer.lookup(targetKey) → Promise<lookupResult>

Iterative XOR-routed walk to find targetKey (typically a peer ID, but any 264-bit key works).

```

const r = await peer.lookup(BigInt('0x' + targetHex));
// {
//   found: true,
//   hops: 3,
//   time: 47, // ms; live RTT since v1.1.2
//   path: [<bigint>, <bigint>, ...], // the hop sequence
// }

```

targetKey may be a BigInt or a 66-char hex string — topicToBigInt converts internally.

peer.health() → healthObj

A cheap, synchronous diagnostic snapshot — safe to poll on a UI tick or from a status endpoint:

```

peer.health();
// {
//   nodeId: '<66 hex>',
//   synaptomeSize: 6, // routing-table entries
//   peers: [<66 hex>', ...], // same, as a list
//   subscriptions: 2,
//   axonRoles: [{ topic, isRoot, children, cacheSize }],
//   wireVersion: '1.0',
//   started: true,
//   transport: { // null on sim/node; populated on web
//     boundCount: 9, // peers authenticated via axona/4
//     meshChannels: 8, // WebRTC data channels (any state)
//     meshOpen: 8, // open data channels
//     meshBound: 8, // open AND axona/4-authenticated
//     bridgeState: 'open',
//   },
//   meshDegraded: false, // true => channels open but NOT bound
// }

```



```
// }
```

meshDegraded is the **routing-truth** signal: true means data channels are open but the axona/4 handshake hasn't authenticated them, so no verified routing is flowing. A single true tick can be a normal mid-handshake transient; treat a value that stays true across several polls as the real signal. (boundCount/meshBound are the honest "usable peers" counts — distinct from the raw channel count.)

```
peer.onLog(level, handler) → () => void
```

Subscribe to structured log events at a given level. **The level comes first**; returns an unsubscribe function.

```
const off = peer.onLog('warn', (msg, ctx) => console.warn(msg, ctx));  
// level ∈ 'debug' | 'info' | 'warn' | 'error'
```

```
peer.onError(handler) / peer.onUpgradeRequired(handler) → () => void
```

onError fires on background AxonaError emissions; onUpgradeRequired fires (and also fan-routes through onError) on a wire-version mismatch, carrying the UpgradeRequiredError with { reason, serverVersion, clientVersion, downloadUrl }.

```
peer.onError((err) => {}); // err = AxonaError  
peer.onUpgradeRequired((err) => showUpgradeBanner(err.context.downloadUrl));
```

6. Topic IDs

Topic IDs share the same 264-bit address space as node IDs. The top 8 bits encode an S2 region; the bottom 256 bits are sha256(canonical input).

```
deriveTopicId(publisherId, topicName) → Promise<string>
```

Compute the 66-char hex topic ID.

publisherId	Mode	Top 8 bits	Hash input
null or undefined or ''	Public	00 (global bucket)	topicName
'<66 hex>' (real nodeId)	Publisher-keyed	publisher's S2 prefix	publisherId + ':' + topicName
'<s2 hex><64 zeros>'	Region-keyed	region's S2 prefix	synth + ':' + topicName

The kernel **doesn't validate** that a 66-char synthetic ID corresponds to a real peer — it's just used as the hash prefix. Use this to bind topics to a region without tying them to a specific publisher.

```
import { deriveTopicId, geoCellId } from '@axona/protocol';  
  
// Public mode  
const pub = await deriveTopicId(null, 'world-news');  
// '00' + sha256('world-news')  
  
// Publisher-keyed  
const pk = await deriveTopicId(myIdHex, 'posts');  
// myS2 + sha256(myIdHex + ':posts')  
  
// Region-keyed (recommended for most chat / forum / feed apps)  
const s2 = geoCellId(lat, lng, 8);  
const synth = s2.toString(16).padStart(2, '0') + '0'.repeat(64);
```

```
const rk = await deriveTopicId(synth, 'world-news');
// regionS2 + sha256(synth + ':world-news')
```

Throws `TypeError` if `topicName` is empty or `publisherId` is non-null and not 66-char hex.

Other `pubsub/post.js` exports

```
makePost({ topic, content, references?, identity?, sign? }) // → SignedPost
verifyPostHash(post) // → boolean
verifyTopicOwnership(post) // → boolean
verifySignature(post) // → Promise<boolean>
canonical(post) // → string (JSON-canonicalized)
sha256Hex(str) // → Promise<string>
```

These are lower-level helpers for protocol implementors building on top of the post-layer rather than `peer.pub`. Most apps don't need them.

7. Envelopes + verification

```
buildEnvelope({ topic, message, ts?, identity, sign? }) → Promise<Envelope>
```

Construct an envelope explicitly. Useful when you want to sign a DM payload before sending it via `peer.send`, or to test verification flows.

```
const env = await buildEnvelope({
  topic: 'dm:to-bob',
  message: { text: 'hi' },
  identity, sign: true,
});
// env = { msgId, ts, topic, message, signerPubkey, signature }
```

Throws `TypeError` if `identity` is missing `privateKey` or `pubkeyHex` when `sign: true`.

```
verifyEnvelope(envelope) → Promise<{ ok, reason?, signed }>
```

Verify that `envelope.signature` is valid for the (`envelope.topic`, `envelope.ts`, `envelope.message`) canonical inputs against `envelope.signerPubkey`, and that `msgId` matches.

Returns a result object, not a boolean — check `.ok`:

```
const res = await verifyEnvelope(env);
if (!res.ok) console.warn('forged/invalid envelope', env.msgId, res.reason);
// res.signed === false for an unsigned envelope that is otherwise valid
// (res.ok === true). Your application decides whether to accept unsigned.
```

> A common mistake: if `!(await verifyEnvelope(env))` is **always > false** — the return value is an object (truthy). Always test `.ok`.

`reason` (when `!ok`) is one of `not_an_object` / `missing_*` / `unknown_signature_scheme` / `wrong_key_or_signature_length` / `bad_signature` / `bad_msgid`. As of kernel v2.7.0, root axons also verify the publisher signature at ingress before fan-out (finding B-4), so spoofed-signature traffic is dropped network-side — but apps should still verify what they consume.

```
computeMsgId({ topic, ts, message, publisher? }) → Promise<string>
```

Stand-alone `msgId` computation matching what `buildEnvelope` does. Useful for verifying a `msgId` server-side.

```
const msgId = await computeMsgId({
  topic: env.topic, ts: env.ts, message: env.message,
```

```
});
console.log(msgId === env.msgId); // true
```

8. Errors

All thrown errors inherit from `AxonaError`:

```
class AxonaError extends Error {
  code: string;           // stable UPPER_SNAKE identifier
  cause?: Error;          // wrapped underlying error
  context?: object;       // machine-readable details
}
```

Subclasses:

Class	Example codes
<code>IdentityError</code>	<code>IDENTITY_INVALID</code> , <code>IDENTITY_LOAD_FAILED</code> , <code>IDENTITY_KEY_IMPORT_FAILED</code>
<code>TransportError</code>	<code>TRANSPORT_NOT_STARTED</code> , <code>TRANSPORT_TIMEOUT</code> , <code>TRANSPORT_NOT_BOUND</code> , <code>TRANSPORT_CLOSED</code>
<code>PublishError</code>	<code>PUBLISH_INVALID_TOPIC</code> , <code>PUBLISH_SIGN_FAILED</code> , <code>PUBLISH_PAYLOAD_TOO_LARGE</code>
<code>SubscribeError</code>	<code>SUBSCRIBE_INVALID_TOPIC</code> , <code>SUBSCRIBE_HANDLER_MISSING</code>
<code>PullError</code>	<code>PULL_TIMEOUT</code> , <code>PULL_MALFORMED_MSGID</code>
<code>MetricsError</code>	<code>METRICS_TIMEOUT</code>
<code>UpgradeRequiredError</code>	<code>UPGRADE_REQUIRED</code> (bridge close code 4426)

The full code list is in `ErrorCodes`:

```
import { ErrorCodes } from '@axona/protocol';
console.log(ErrorCodes.PUBLISH_SIGN_FAILED); // 'PUBLISH_SIGN_FAILED'
```

Switch on `.code`, not `.message`:

```
try { await peer.pub(topic, msg); }
catch (err) {
  if (err.code === ErrorCodes.PUBLISH_SIGN_FAILED) {
    await reload();
  } else { throw err; }
}
```

Wire-format helpers

```
isWireError(obj)      // → boolean – does this look like an over-the-wire AxonaError?
fromWire(obj)         // → AxonaError – reconstruct
err.toWire()          // → { __axonaError: true, class, code, message, context }
```

Errors thrown inside remote handlers (i.e., inside another peer's `onMessage` reached via `peer.send`) survive serialization with their class and code intact.

9. Persistence

PersistenceAdapter (interface)

```
class PersistenceAdapter {
  async load(key) { /* returns previously-saved string or null */ }
  async save(key, value) { /* persists; value is string */ }
  async clear() { /* wipe everything; idempotent */ }
}
```

Implement this to plug in your own storage (Redis, S3, postgres, etc.). Two built-in implementations:

`createIndexedDbPersistence({ dbName })` → `PersistenceAdapter`

Browser. Uses a single object store inside the named DB.

```
import { createIndexedDbPersistence } from '@axona/protocol';

const peer = new AxonaPeer({
  domain, node, identity,
  persistence: createIndexedDbPersistence({ dbName: 'my-app' }),
});
```

`createFilePersistence({ path })` → `PersistenceAdapter`

Node. Single JSON file on disk.

```
import { createFilePersistence } from '@axona/protocol';

const peer = new AxonaPeer({
  domain, node, identity,
  persistence: createFilePersistence({ path: './state.json' }),
});
```

Manual snapshots

```
const state = await peer.snapshot();
// state = { identity?, subscriptions, synaptome, lookups, ... }
// – any JSON-safe object

await peer.fromSnapshot(state);

// static factory:
const peer = await AxonaPeer.fromSnapshot(state, { engine, node, transport });
```

Transport + protocol surface

10. Contracts

Abstract base classes that transports / DHT implementations extend. Application developers rarely use these directly — they're for authoring custom transports or alternate DHT engines.

Transport (abstract class)

```
import { Transport } from '@axona/protocol';

class MyTransport extends Transport {
  async start(localNodeId) { /* ... */ }
  async stop() { /* ... */ }
  async send(peerId, type, body) { /* ... */ } // request/response
}
```

```

    async notify(peerId, type, body) { /* ... */ } // fire-and-forget
    async openConnection(peerId)    { /* ... */ }
    async closeConnection(peerId)   { /* ... */ }
    isConnected(peerId)             { /* ... */ }
    getLatency(peerId)              { /* RTT ms or -1 */ }
    onRequest(type, handler)        { /* ... */ }
    onNotification(type, handler)    { /* ... */ }
    onPeerDied(handler)             { /* ... */ }
  }

```

The full contract surface is in `@axona/protocol/contracts/Transport.js` (sub-path import).

DHT (**abstract class**)

Used by AxonaManager's dht adapter. Defines `getSelfId`, `findKClosest`, `routeMessage`, `sendDirect`, `onRoutedMessage`, `onDirectMessage`. See `axona-peer/src/browser_engine.js` for a concrete adapter that wraps AxonaPeer's methods.

BootstrapService (**abstract class**)

For implementing peer discovery (signaling brokers, seed lists, DNS-SD, etc). Rarely needed by app code.

11. Sim transport

In-process router for testing + simulators.

```
new SimNetwork({ latencyFn? })
```

```

import { SimNetwork } from '@axona/protocol';

// All edges = 0ms
const net = new SimNetwork();

// 50ms each way
const net = new SimNetwork({ latencyFn: () => 50 });

// Custom per-pair
const net = new SimNetwork({
  latencyFn: (fromId, toId) => Math.abs(hash(fromId) - hash(toId)) % 100,
});

```

```
simTransport({ network, identity, heartbeatMs?, ... }) → SimTransport
```

Factory that builds + wires a `SimTransport` connected to a `SimNetwork`. Same constructor options as `new SimTransport({ ... })` but with a more concise call site.

```

import { SimNetwork, simTransport } from '@axona/protocol';

const network = new SimNetwork();
const t = simTransport({ network, identity, heartbeatMs: 0 });
await t.start(identity.id);
await t.openConnection(otherIdentity.id);

```

`heartbeatMs: 0` disables heartbeats (recommended for tests; the default heartbeat is helpful for production where peers may go away silently, useless for in-process synchronous flows).

11b. Web transport (browser)

The production browser transport: a WebSocket to the bridge for bootstrap + signaling, and a WebRTC mesh for direct peer-to-peer traffic. It owns the bridge socket lifecycle (reconnect/backoff, ping/pong, stale detection), the version-gate handshake, and the axona/4 authenticated-identity handshake on both the bridge link and every mesh link.

webTransport(opts) → CompositeTransport

```
import { webTransport } from '@axona/protocol/transport/web/index.js';

const transport = webTransport({
  bridgeUrl: 'wss://bridge.axona.net', // required
  identity, // from deriveIdentity() – signs the handshake
  peerVersion: '3.11.0', // your app version (gated by the bridge)
  reconnect: true, // auto-reconnect with backoff (default)
  // log, autoHandshake, handshakeTimeoutMs, pingIntervalMs,
  // reconnectInitialMs, reconnectMaxMs, WebSocketImpl ...
});
await transport.start(); // resolves after the bridge handshake
```

Beyond the Transport contract (send/notify/onRequest/onNotification/boundPeers/onPeerBound/start/stop), it exposes an observability surface for the UI:

```
transport.bridgeState // 'connecting'|'open'|'stale'|'disconnected'|'upgrade-required'
transport.bridgeInfo // { connId, version, kernelVersion, turn } | null
transport.bridgeRtt // last ping→pong ms (null until first pong)
transport.bridgeRttAvg // mean of the recent RTT window
transport.bridgeNodeId // bridge's 66-hex nodeId (null pre-handshake)
transport.onBridgeState((state, detail) => {}); // → unsub
transport.onWelcome((info) => {}); // → unsub (replays last)
transport.onPingTraffic((dir) => {}); // 'sent' | 'recv'
transport.reconnectNow(); // force an immediate reconnect (tab resume)
transport.boundPeers(); // bigint[] – axona/4-authenticated peers
```

Channel binding (finding A-1): each mesh link's axona/4 proof is bound to that connection's DTLS certificate fingerprint, so a fingerprint-rewriting bridge cannot transparently MITM "direct" peer traffic. The bridge is trusted for signaling only, never for the contents of peer-to-peer links. See §17.

12. Handshake

Wire-version handshake helpers used by transports on a fresh signaling channel. Most app code never touches these — they're plumbed into simTransport/webTransport/etc. internally.

```
buildClientHello({ version, nodeId }) // → frame for client → server
buildServerHello({ version, minVersion }) // → frame for server → client
parseHello(frame) // → { version, nodeId? }
parseVersion(str) // → { major, minor, patch }
compareVersions(a, b) // → -1 | 0 | 1
wireCompatible(peerVersion, minRequired) // → boolean
performClientHandshake(channel, opts) // → handshake result
performServerHandshake(channel, opts)
```

The constants WIRE_VERSION, KERNEL_VERSION, UPGRADE_CLOSE_CODE are exported from the same module — see §16.

Low-level utilities

13. Ed25519 helpers

Web Crypto Ed25519 wrappers. Useful when you want to sign / verify outside of buildEnvelope — e.g. for signing direct-message payloads manually.

```
const { pubkey, privateKey } = await generateKeyPair();
// pubkey = Uint8Array(32); privateKey = CryptoKey

const pubHex = await exportPublicKey(pubkey);
const pubKey = await importPublicKey(pubHex);

const sig = await sign(privateKey, dataBytes); // Uint8Array(64)
const ok = await verify(pubKey, sig, dataBytes);

const signer = makeSigner(privateKey); // (bytes) => Promise<sig>
const verifier = makeVerifier(pubKey); // (sig, bytes) => Promise<boolean>
```

Implementation-agnostic: substitute @noble/ed25519 for runtimes that don't have Web Crypto Ed25519 (Chrome <110, Safari <17, Firefox <130, Node <20). The makeSigner / makeVerifier contract is what the rest of the kernel binds against.

14. Hex / ID math

```
ID_BITS // 264
HASH_BITS // 256
S2_BITS // 8
HEX_CHARS // 66
MAX_ID // BigInt (2**264 - 1)
MAX_HASH // BigInt (2**256 - 1)
MAX_S2 // 255

toHex(bigint) // → 66-char lowercase hex
fromHex(hex) // → BigInt
isHexId(s) // → boolean (66-char lowercase hex)
assembleId(s2Prefix, hash) // → BigInt — concat 8-bit prefix + 256-bit hash
extractS2Prefix(idBig) // → number 0..255
extractHash(idBig) // → BigInt (bottom 256 bits)
s2PrefixOfHex(hex) // → number 0..255

xorDistance(a, b) // → BigInt
stratumOf(a, b) // → 0..264 — count of leading zero bits
clz264(bigint) // → 0..264 — count of leading zero bits in a 264-bit value
randomU256() // → BigInt — cryptographically random
```

These are useful when implementing custom routing logic, K-closest overrides, or analytics on top of the address space.

15. Geo utilities

```
geoCellId(lat, lng, bits) // → integer — S2 cell ID at the given bit depth
haversine(lat1, lng1, lat2, lng2) // → km between two points
roundTripLatency(lat1, lng1, lat2, lng2) // → ms — rough fiber estimate
randomU32() // → 0..2**32 - 1
```

Sub-path import for the remaining geo helpers (@axona/protocol/utils/geo.js): getRegion, getContinent, several XOR routing-table builders. Rarely needed by app code.

16. Constants

```

WIRE_VERSION      // '1.0'      - wire format major.minor
KERNEL_VERSION    // '2.8.0'    - kernel semver
AUTH_PROTO        // 'axona/4'  - authenticated-identity handshake tag
UPGRADE_CLOSE_CODE // 4426      - WebSocket close code for version mismatch
ID_BITS           // 264
HEX_CHARS         // 66

```

WIRE_VERSION is what bridges enforce gates against; KERNEL_VERSION is informational and corresponds to the npm release tag.

17. Security model (what the protocol guarantees)

The full, versioned record is the security changelog; this is the developer-facing summary of what's enforced as of kernel v2.8.0. Everything here is **self-authenticating** — no certificate authority, no central trust server, no reputation service.

- **Authenticated identity (axona/4).** A node's bottom 256 bits are SHA-256(pubkey), and every connection runs a handshake proving the peer holds that key (BIND: pubkey hashes to the id · POSSESS: Ed25519 signature · CHANNEL: the signature is bound to this live connection so a captured proof can't be replayed onto another link). A peer cannot claim an id it doesn't own.

- **Channel binding (untrusted bridge).** Mesh links fold each side's DTLS certificate fingerprint into the signed handshake, so a bridge that terminates DTLS to interpose on a "direct" link produces divergent bindings and the handshake fails. The bridge is trusted for *signaling only*.

- **Pub/sub trust boundary.** A peer can only subscribe *itself* (every path checks the authenticated sender, not a payload-named id) — no reflection/amplification at a victim. A node only hosts topics it's among the K-closest to — no memory-exhaustion via random-topic floods. Publisher signatures are verified at root-axon ingress before fan-out.

- **Verified routing admission (eclipse resistance).** Routing-table entries are *earned*: a peer named in routing gossip is a candidate, admitted only after this node connects and the peer authenticates via axona/4. Forged gossip cannot inject hops. (Observe this via `health().meshDegraded` and `boundCount`.)

- **Key hygiene.** Browser identities can hold a **non-extractable** signing key (`deriveIdentity({ extractable: false })`); `loadIdentity` verifies the private key matches its public key on load.

What the protocol does **not** do: encrypt your application payload (it's opaque bytes — encrypt inside message if you need confidentiality), or vouch for what an authenticated peer says beyond what cryptography proves. Open security work is tracked privately; the public changelog lists only resolved items.

A word on stability

@axona/protocol v2.x is stable for the application surface documented in §§1–9. The transport + protocol surface (§§10–12, incl. the browser webTransport and the axona/4 security model in §17) is also stable but expect occasional additions (new transport factories, new bootstrap services). Low-level utilities (§§13–16) may grow but won't change incompatibly.

The AxonaPeer constructor still accepts a legacy engine argument for backward compatibility with pre-Phase-5 callers; new code should pass domain instead.

For changelog and migration notes between minor versions, see <<https://github.com/axona-net/axona-protocol/releases>>.